

Policy: A function named `get` should return only on success

Document #: D3857R0
Date: 2025-10-02 19:46 EDT
Project: Programming Language C++
Audience: LEWG
Reply-to: Pablo Halpern
<phalpern@halpernwrightsoftware.com>

Contents

- 1 [Abstract](#)
- 2 [Change Log](#)
- 3 [Introduction](#)
- 4 [Proposals](#)
 - 4.1 [Proposed change to P3091](#)
 - 4.2 [Proposed Policy Wording](#)
- 5 [References](#)

§ 1 Abstract

Should a value-retrieval function that returns a disengaged `optional` on lookup failure be called `get`? This question was briefly debated in discussion of [\[P3091R4\]](#), which introduces just such a function. In this paper, I try to take a principled approach to this question. My conclusion is that a successful return from `get` should imply a successful retrieval of a value, and that a function that indicates failure through the normal return value should be called something else. This paper makes two, somewhat independent, proposals: 1) Rename the `get` functions in P3091 to `lookup`, and 2) have a general LEWG policy for using `get` as a function name.

§ 2 Change Log

R0 (2025-10 pre-Kona mailing)

— Initial version

§ 3 Introduction

[P3091R3] was approved by LEWG during the June 2025 meeting in Sofia. It proposes new `get` member functions in `std::map`, `std::unordered_map`, and `std::flat_map`, which return a `std::optional<value_type&>`. The return value is engaged if the specified key is found within the map and disengaged otherwise.

The use of a special, “empty”, return value (e.g., a null pointer, disengaged `optional`, or erroneous expected) is a break from the typical pattern for `get` functions, both within the Standard Library and in the wider programming world. Within the Working Paper, member and nonmember `get` functions for `unique_ptr`, `shared_ptr`, `tuple`, `variant`, `reference_wrapper`, `subrange`, `basic_format_string`, `basic_format_args`¹, and `future` do not return values that can indicate failure; either they can never fail when called within contract or they indicate failure other than by a normal return (e.g., they exit via an exception). The understood meaning of `get` in all of these contexts is “retrieve something that the user believes is retrievable.” The only exceptions to this interface rule within the Standard Library are in `locale` facets and `istream`, where the retrieved value is not communicated via the return value at all, but rather via a pass-by-reference parameter. The idiosyncratic uses of `get` in `locale` and `istream` are arguably poor exemplars for designing new interfaces.

It was pointed out in the LEWG discussion of [P3091R4] that the proposed interface for `get` is inconsistent with other uses of `get`. Although `unique_ptr::get` and `shared_ptr::get` sometimes return `nullptr`, it was pointed out that the null value in these cases indicate a *successful* retrieval of a value that happens to be a null pointer, and is thus unrelated to the use of `optional` in P3091. At the time, we had not performed thorough review of `get` in the working paper, and minimal time was allocated to the discussion in a reasonable attempt to avoid endless bike-shedding. An LEWG poll in Sophia on 2025-06-19 kept the status-quo as of [P3091R3]:

POLL: Rename `.get` to `.lookup`

SF	F	N	A	SA
1	3	6	2	6

The consensus against the change was tepid, at best. Furthermore, based on subsequent discussions and the research presented above, I, as the author of P3091, would change my vote from neutral to strongly in favor. The [Proposals](#) below would reverse this decision for P3091 and establish a general naming policy, in the hopes of avoiding unintuitive and inconsistent interfaces in the Standard.

§ 4 Proposals

This paper contains two proposals that can, and should, be polled independently. In particular, the proposed change to P3091 can be accepted even if the formal policy is not

accepted or is deferred for modification and further discussion.

§ 4.1 Proposed change to P3091

Rename the new `get` member functions for map-like containers in the latest version of P3091 to `lookup`. Note: [P3091R4] was forwarded to LWG at the Sophia meeting, but has not been reviewed as of this date).

§ 4.2 Proposed Policy Wording

When adding a new function, the name `get` shall be reserved for a function that retrieves an object (or reference to an object) and whose normal (non-exceptional, non-terminating) return implicitly indicates that the object was successfully retrieved. A function that potentially returns a not-found or not-available status, regardless of whether such a status indicates an error, should be named something other than `get` (e.g., `lookup`, `fetch`, `get_if`, etc.).

§ 5 References

[P3091R3] Pablo Halpern. 2024-10-14. Better lookups for ``map`` and ``unordered_map``.

<https://wg21.link/p3091r3>

[P3091R4] Pablo Halpern. 2025-06-20. Better lookups for ``map``, ``unordered_map``, and ``flat_map``.

<https://wg21.link/p3091r4>

-
1. A call to `basic_format_args<C>::get(i)` returns a default-constructed object if `i` is larger than the number of arguments specified on construction, but that does not indicate a failed lookup. ↩